```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    mean_squared_error, r2_score
)
df = pd.read_csv('diabetes.csv')
print("Dataset shape:", df.shape)
print(df.head())
```

```
Dataset shape: (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

```
zero_invalid_cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[zero_invalid_cols] = df[zero_invalid_cols].replace(0, np.nan)

for col in zero_invalid_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

```
/tmp/ipykernel_924/3308215050.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df[col].fillna(df[col].median(), inplace=True)
```

```
feature_cols = [
    "Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI", "DiabetesPedigreeFunction", "Age"
]

X_all = df[feature_cols].values
```

```
y_clf = df["Outcome"].values
```

```
target_reg = "BMI"
feature_cols_reg = [c for c in feature_cols if c != target_reg]

X_reg = df[feature_cols_reg].values
y_reg = df[target_reg].values

print("Regression target column:", target_reg)
print("Regression feature columns:", feature_cols_reg)
```

```
Regression target column: BMI
Regression feature columns: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'DiabetesPedigreeFunction',
```

```
X_train_clf, X_test_clf, y_clf_train, y_clf_test = train_test_split(
    X_all, y_clf, test_size=0.3, random_state=42, stratify=y_clf
)

# Regression split
X_train_reg, X_test_reg, y_reg_train, y_reg_test = train_test_split(
    X_reg, y_reg, test_size=0.3, random_state=42
)
scaler_clf = StandardScaler()
```

```
X_train_clf_scaled = scaler_clf.fit_transform(X_train_clf)
X_test_clf_scaled = scaler_clf.transform(X_test_clf)

scaler_reg = StandardScaler()
X_train_reg_scaled = scaler_reg.fit_transform(X_train_reg)
X_test_reg_scaled = scaler_reg.transform(X_test_reg)
```

```
comparison = pd.DataFrame({
    "Original Mean": X_train_clf.mean(axis=0),
    "Original Std": X_train_clf.std(axis=0),
    "Scaled Mean": X_train_clf_scaled.mean(axis=0),
    "Scaled Std": X_train_clf_scaled.std(axis=0)
})
print(comparison)
```

```
   Original Mean  Original Std   Scaled Mean  Scaled Std
0       3.882682      3.346976  4.202101e-17         1.0
1     121.564246     29.923802  1.385194e-17         1.0
2      72.227188     12.175339 -2.299009e-16         1.0
3      28.886406      8.251412  2.170569e-16         1.0
4     138.057728     79.653266 -2.047943e-16         1.0
5      32.615829      6.724421  4.258956e-15         1.0
6       0.480529      0.334941 -5.520103e-16         1.0
7      33.551210     11.783652 -2.260761e-16         1.0
```

```
k = 5
```

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_clf_scaled, y_clf_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
y_pred = knn.predict(X_test_clf_scaled)
print("Predictions:")
print(y_pred[:20])
```

```
Predictions:
[0 0 1 1 0 0 1 0 0 0 0 0 0 1 1 1 1 0 1 0]
```

```
y_prob = knn.predict_proba(X_test_clf_scaled)[:, 1]
print(y_prob[:20])
```

```
[0.  0.2 0.8 0.6 0.4 0.2 0.8 0.  0.  0.2 0.  0.  0.2 0.8 0.8 0.8 0.6 0.
 0.8 0.2]
```

```
accuracy = accuracy_score(y_clf_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
```

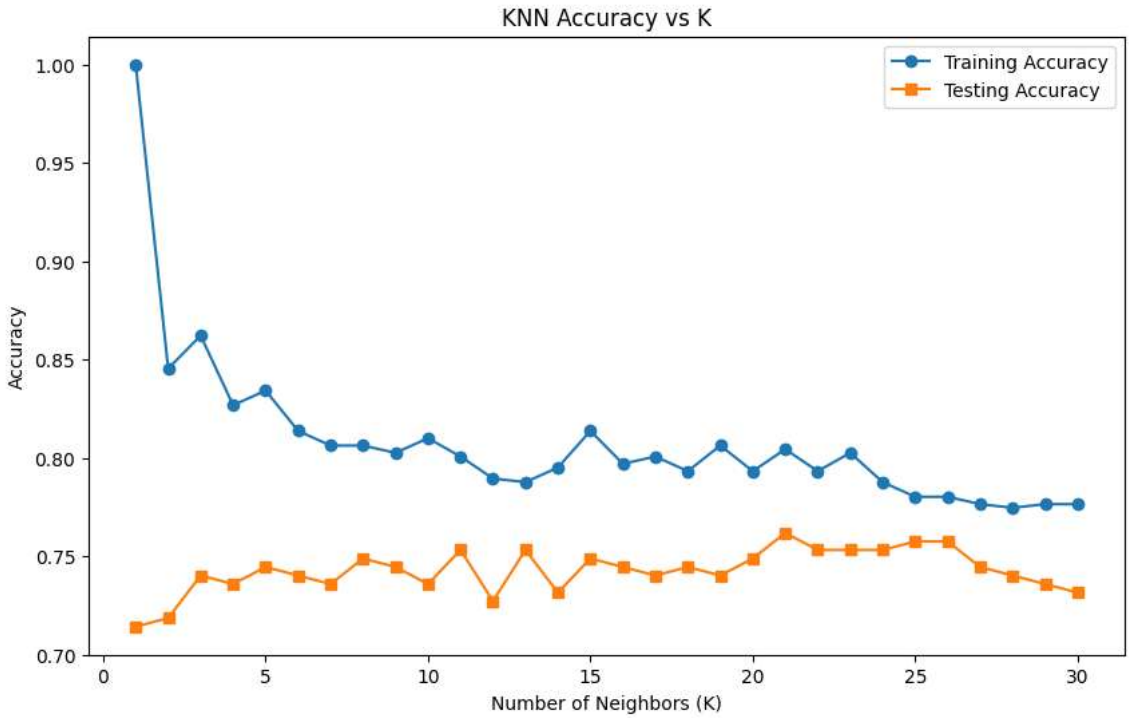
```
Accuracy: 0.7446
```

```
k_values = range(1, 31)
train_accuracy = []
test_accuracy = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_clf_scaled, y_clf_train)

    train_accuracy.append(model.score(X_train_clf_scaled, y_clf_train))
    test_accuracy.append(model.score(X_test_clf_scaled, y_clf_test))
```

```
plt.figure(figsize=(10, 6))
plt.plot(k_values, train_accuracy, marker="o", label="Training Accuracy")
plt.plot(k_values, test_accuracy, marker="s", label="Testing Accuracy")
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy vs K")
plt.legend()
plt.show()
```




```
best_k_accuracy = k_values[np.argmax(test_accuracy)]
best_accuracy = max(test_accuracy)
print("Best K :",best_k_accuracy)
print("Best Test Accuracy :",best_accuracy)
```

```
Best K : 21
Best Test Accuracy : 0.7619047619047619
```

```
from sklearn.model_selection import cross_val_score

cv_scores = []
for k in range(1, 31):
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train_clf_scaled, y_clf_train, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())
```

```
best_k_cv = np.argmax(cv_scores) + 1
print("Best K based on Cross Validation:", best_k_cv)
print("Best CV Accuracy:", max(cv_scores))
```

```
Best K based on Cross Validation: 12
Best CV Accuracy: 0.7561613014884043
```

```
param_grid = {
    "n_neighbors": list(range(3, 31, 2)),
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan", "minkowski"],
    "p": [1, 2]
}
```

```
# Initialize and fit GridSearchCV
grid_search = GridSearchCV(
    estimator=KNeighborsClassifier(),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train_clf_scaled, y_clf_train)

# Print best parameters and best score
print("Best Hyperparameters:", grid_search.best_params_)
print("Best CV Accuracy:", grid_search.best_score_)
```

```
# Evaluate best model on test set
best_knn = grid_search.best_estimator_
test_acc = best_knn.score(X_test_clf_scaled, y_clf_test)
print(f"Test Set Accuracy with Best Parameters: {test_acc:.4f}")
```

```
Best Hyperparameters: {'metric': 'euclidean', 'n_neighbors': 11, 'p': 1, 'weights': 'uniform'}
Best CV Accuracy: 0.7543094496365523
Test Set Accuracy with Best Parameters: 0.7532
```

```
Start coding or generate with AI.
```